

MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ITWM106

Mobile Application Development 3 (cross-platform)

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

Basic React Components

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

Learning Objectives

- At the end of this lecture, students should be able to:
- Explain what a component is in React Native.
- Identify common React Native core components.
- Use View, Text, Image, TextInput, Button, Pressable, and ScrollView.

What Is a Component?

- A component is a reusable building block of a user interface.
- Think about a mobile application such as a shopping app.
- A product screen might contain:



What Is a Component?

- This interface can be broken into several components:
- ProductScreen
 - Image
 - Text
 - Text
 - Button

Instead of creating the entire screen as one large block of code, React Native allows us to build the interface using smaller components.

What Is a React Native Component?

- A React Native component is a JavaScript function that returns a description of what should appear on the screen, Example:

```
import { Text } from 'react-native';

export default function App() {
  return (
    <Text>Hello React Native!</Text>
  );
}
```

Why Do We Use Components?

- **Reusability**
 - A component can be used multiple times.
- **Maintainability**
 - Components make applications easier to maintain
- **Organization**
 - Components allow developers to divide a complicated interface into smaller parts.
- **Consistency**
 - If the same component is used throughout an application, the design and behavior remain consistent.



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph [@mseufCCMS](https://www.facebook.com/mseufCCMS) www.mseuf.edu.ph

Basic Components

- View
- Text
- Image
- Lists
- Button
- TextInput
- Scrollviews
- Pressable and Touchables
- SafeAreaView



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph [@mseufCCMS](https://www.facebook.com/mseufCCMS) www.mseuf.edu.ph

View

- View is the most common element in React Native.
- You can consider it as an equivalent of the div element used in web development.
- It provides a container for other components.
- The View can be assumed as a default element in React Native.

Basic Syntax

```
<View></View>
```

View

```
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View>
      <Text>Hello</Text>
      <Text>Welcome to React Native</Text>
    </View>
  );
}
```

View Use Cases

- When you need to wrap your elements inside the **container**, you can use View as a container element.
- When you want to nest more elements inside the parent element, both parent and child can be View.
- It can have as many children as you want.
- When you want to style different elements, you can place them inside View since it supports style property, flexbox etc.
- View also supports synthetic touch events, which can be useful for different purposes.

SafeAreaView

- SafeAreaView is a component that provides a way to ensure that your content is displayed within the safe areas of a device's screen, particularly in modern smartphones like the iPhone X and later models.
- These devices have rounded corners and a notch at the top, which can obscure parts of your app's content if not properly accounted for
- The safe area refers to the portion of the screen where your app's content can be safely displayed without being obstructed by system elements like the notch, status bar, or rounded corners.

SafeAreaView

- To use SafeAreaView, you wrap your content or components inside it. Here's a basic example:

```
<SafeAreaView>
  <View>
    <Text>Your app content goes here.</Text>
  </View>
</SafeAreaView>
```

Text

- A React component for displaying text.
- Text supports nesting, styling, and touch handling.

Basic Syntax:

```
<Text>Hello, world!</Text>
```

Image

- A React component for displaying different types of images, including **network images, static resources, temporary local images, and images from local disk**, such as the camera roll.

Static Image Resources

- React Native provides a unified way of managing images and other media assets in your Android and iOS apps.
- To add a static image to your app, place it somewhere in your source code tree and reference it like this:

```
<Image source={require('./my-icon.png')} />
```

Network Images

- Many of the images you will display in your app will not be available at compile time, or you will want to load some dynamically to keep the binary size down.
- Unlike with static resources, you will need to manually specify the **dimensions of your image**. It's highly recommended that you use https as well in order to satisfy App Transport Security requirements on iOS

```
<Image source={{
  uri: 'https://reactnative.dev/docs/assets/p_cat2.png',
}}
style={{ width: 200, height: 200 }}
/>
```



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

Button

- Buttons are touchable elements used to interact with the screen and to perform an operation.
- Supports a minimal level of customization.
- If this button doesn't look right for your app, you can build your own button using Pressable, TouchableOpacity or TouchableWithoutFeedback.

```
<Button title="Click Me"
  onPress={() => alert("Button Pressed!", "Welcome to React
Native.")} />
```



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

Pressable

- Pressable allows us to create a customizable touchable element.
- Unlike Button, we can completely control the contents and styling
- Example:

```
import { Pressable, Text } from 'react-native';
```

```
<Pressable
  onPress={() => alert('Pressed!')}
>
  <Text>Click Me</Text>
</Pressable>
```



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

TouchableOpacity

- TouchableOpacity is a component that provides **touchable behavior** to its child components.
- It's part of the React Native core and is used to make elements on your app's user interface **respond to touch events**, such as taps.
- TouchableOpacity is a common choice for creating **interactive buttons** and elements in React Native applications
- TouchableOpacity wraps its child components and makes them respond to touch events, such as presses and releases.
- When a user taps the TouchableOpacity, it reduces the opacity of its child components, creating a visual feedback effect to indicate that the touch event has been recognized.



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

TouchableOpacity

```
<TouchableOpacity  
  onPress={() => alert('Pressed')}  
>  
  <Text>Press Me</Text>  
</TouchableOpacity>
```

ScrollView

- ScrollView allows the user to scroll through content.
- Example:

```
<ScrollView>  
  <Text>Student 1</Text>  
  <Text>Student 2</Text>  
  <Text>Student 3</Text>  
  <Text>Student 4</Text>  
  <Text>Student 5</Text>  
</ScrollView>
```

ScrollView

- By default, the `<ScrollView>` provides vertical scrolling.
- For horizontal scrolling:
`<ScrollView horizontal>`
- ScrollView renders all of its children.
- Therefore, it is appropriate for relatively small amounts of content.
- For large datasets we can use FlatList, which is designed for efficient rendering of lists.

Styling Elements

- There are a couple of ways to style your elements in React Native.
- You can use the **style** property to add the styles inline.
- However, this is not the best practice because it can be hard to read the code.
- All of the core components accept a prop named style.
- The style names and values usually match how CSS works on the web, except names are written using **camel casing**, e.g. **backgroundColor** rather than **background-color**.

Styling Elements

- The style prop can be a plain old JavaScript object. That's what we usually use for example code. You can also pass an array of styles - the last style in the array has precedence, so you can use this to inherit styles.

```
<Text style={{ color: 'blue', fontWeight: 'bold', fontSize: 30, }} >Hello World</Text>
```

Styling Elements

- As a component grows in complexity, it is often cleaner to use `StyleSheet.create` to define several styles in one place.

```
const styles = StyleSheet.create({
  textStyle: {
    color: 'blue',
    fontWeight: 'bold',
    fontSize: 30,
  }
});
```

Styling Text

Property

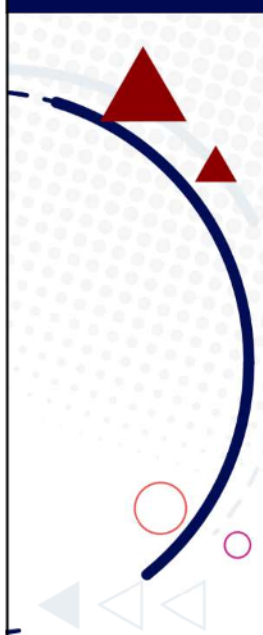
fontSize
fontWeight
textAlign
fontStyle
lineHeight
numberOfLines

Purpose

Controls text size
Controls thickness
Controls horizontal alignment
Controls italic style
Controls line spacing
Limits number of lines




 MANUEL S. ENVERGA UNIVERSITY FOUNDATION
 COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
 CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY




ITWM106

Mobile Application Development 3 (cross-platform)

 ccms@mseuf.edu.ph
  @mseufCCMS
  www.mseuf.edu.ph